

Daily Workflow

One interactive session owns a ticket through eight phases: brainstorm, spec, plan, implement, tests, docs, review, done. Every phase but the light one below writes a file under `.work/tickets/<id>/`, and two of them cannot be skipped by accident — `/crew:implement` refuses without an approved plan, and `/crew:done` refuses without a clean review receipt, a clean verify gate, and a passing completion audit.

For how the scope guard decides what a ticket may touch and what a report looks like when it refuses a write, see [daily-workflow-scope.md](#) (T3).

Full Path — Worked Example

1. Brainstorm. You type:

```
/crew:brainstorm add a --dry-run flag to the migrate command
```

The session mints T-0091, creates `.work/tickets/T-0091/`, and asks **one question at a time**:

Should `--dry-run` also print what it would do to files that already match (no-op writes), or only the ones that would actually change?

You answer. It proposes two approaches, recommendation first, then writes `.work/tickets/T-0091/direction.md` and stops. You say yes.

2. Spec. You type `/crew:spec T-0091`. The session reads the direction, uses `crew:explorer` to pin down `path:line` evidence, and writes `spec.md` with Intent, Exclusions, Evidence, Unknowns, Touch and Acceptance. `.work/INDEX.md` gets a row.

3. Plan. You type `/crew:plan T-0091`. The session reads `spec.md`, writes one step per unit of work in `plan.md` (Files/Test/Risk each), checks every Files: entry against the spec's Touch globs, and enters **plan mode** to show you the whole thing. You review it. On your yes, it asks you to type `/crew:approve T-0091`. crew's prompt hook sees that you typed it and writes the approval receipt, bound to this plan's exact contents — edit `plan.md` after this and the receipt no longer matches.

4. Implement. You type `/crew:implement T-0091`. First thing it does: checks the receipt. If you skipped step 3, or edited the plan after approving it, it refuses here and tells you which. Assuming it passes, it records the scope base (`scope_base.py --record`), works the plan step by step — test first, watch it fail, implement, watch it pass — and the **plan-approval + scope guard hook** blocks any write outside the spec's Touch globs before it happens, not after.

5–6. Tests and docs. Coverage lands as part of implementing the plan's steps. `/crew:docs` runs next and usually says "none" — most tickets touch no document that needs updating.

7. Review. `/crew:implement` calls `/crew:review T-0091` last, after tests and docs. Codex reviews the bundle (or Copilot, or the Claude fallback, whichever survives the author-family strike), reports BLOCK/FIX/NIT lines, and you fix the BLOCKs. Two rounds total, ticket-wide — a third is refused and the ticket becomes `NEEDS_REPLAN`.

8. Done. You type `/crew:done T-0091`. Three checks, all required: the review receipt rebuilds clean, the verify gate is clean, and the completion audit (the whole tree diffed against the scope base) finds nothing outside scope. Any one failing refuses the close and names what to fix. On success it appends a metrics row, marks the ticket done, and clears a stale handoff.

What Each Hook Does, in Order

When	Hook	Does
Every write	plan-approval + scope guard (PreToolUse)	refuses an edit with no approval receipt, or outside the spec's Touch
End of turn	verify gate (Stop)	refuses to end the turn on a red check
End of turn	completion scope audit (Stop)	diffs the whole tree against the scope base, catches shell-made writes too
Session start	context	brief on branch, open ticket, gate state, codemap freshness

None of these ask you anything mid-turn — a blocked write or a red gate shows up as a refusal with a reason, not a prompt.

For a small, well-understood change (one subsystem, no new behaviour, known cause, nothing touching auth/SQL/IaC/secrets/migrations):

```
/crew:fix the changed-file list in implement.md prints nothing on an empty diff
```

Every phase still runs, compressed:

- **Direction:** one line, not a question round — Fix: <what you typed>.
- **Spec:** six lines — Intent, Touch, Acceptance — Exclusions, Evidence and Unknowns omitted, not forgotten.
- **Plan:** exactly one step, still shown, still needing the approval receipt.
- **Implement, tests, docs:** same as the full path, sized to one step.
- **Review:** one Codex round instead of two.
- **Done:** the same three checks — the light path does not skip its own gate.

If the change turns out to be bigger than it looked — a second subsystem shows up, the cause wasn't what you thought — the session says so and hands off to /crew:spec <id> to fill in what the light path left out, rather than forcing the rest of the ticket through a path that no longer fits it.

If Something Refuses

Refusal	Means	Do
/crew:implement says no approved plan	step 3 was skipped, or plan.md changed after approval	/crew:plan <id>, then type /crew:approve <id>
a write is blocked outside Touch	the file isn't in the spec's declared scope	amend spec.md's Touch and re-approve the plan, or don't make the edit
/crew:done reports NEEDS_REPLAN	the review budget (two rounds) is spent	/crew:plan <id> for a successor plan; no third round
/crew:done fails the	a path outside scope changed,	file it to TODO.md, not to this ticket,

Refusal

completion audit

Means

including one a shell command
wrote

Do

then rerun

See [Troubleshooting](#) for the rest.

Scope and Approval

Every crew 1.0 ticket has a contract: the spec says which files it may touch, the plan says which of those each step changes, and you approve both. Two hooks then hold the session to that contract. This section explains what they do, what they print, and how to change scope while you work.

The Contract

A ticket lives in `.work/tickets/<id>/`:

- `spec.md` has six sections: `## Intent`, `## Exclusions`, `## Evidence`, `## Unknowns`, `## Touch` and `## Acceptance checks`. `## Touch` lists one repo-relative glob or path per bullet:

```
## Touch

- `src/export/**`

- `tests/test_export.py`

- `docs/export.md`
```

- `plan.md` is a list of steps. Each step has a `Files:` line, a `Test:` line and a `Risk:` line. Every `Files:` entry must fall inside `Touch`. A plan cannot add a path the spec does not list.

Check the contract at any time:

```
python3 <crew>/hooks/scripts/crew_ticket.py validate --ticket T-0042
```

A plan path outside `Touch` is reported as `INVALID: plan Files entry '...' is outside spec ## Touch`. To fix it, add the path to `Touch`. crew never widens `Touch` for you.

Approval

When the plan is ready, the session asks you to approve it. Type this yourself:

```
/crew:approve T-0042
```

crew's UserPromptSubmit hook sees the prompt you typed, validates the contract, then records the sha256 of spec.md and plan.md in <git-common-dir>/crew/tickets/T-0042/approval.json, marked approved_via: "user-prompt". If the contract does not validate, the hook blocks the prompt and says why. That file is outside the worktree, and the scope guard refuses any Write or Edit to it and any shell command that runs crew_ticket.py approve, so the session cannot approve its own plan. The approval is a step you take, not a lock.

crew_ticket.py status --ticket T-0042 prints one of three states:

- approved: both files match the receipt.
- stale: spec.md or plan.md changed since approval.
- none: the ticket has never been approved.

While You Implement: The Scope Guard

Before every Write, Edit, MultiEdit or NotebookEdit, the guard checks the target file:

- The ticket's own .work/tickets/<id>/ files are always allowed, so you can amend the spec and plan.
- Any other file needs a current approval and a path inside Touch.
- Nothing else is exempt, including .crew/, TODO.md, .claude/ and crew's own policy files. If the ticket changes one of them, list it in Touch.
- Approval receipts, the review ledger and the scope base are always refused.

The guard checks the real file a symlink points to and the path as written, so a link cannot move a write into or out of scope. . . is resolved the way the operating system resolves it. Refusals look like this:

```
SCOPE GUARD: refused Edit on other/keep.py.
```

```
Reason: other/keep.py is outside T-0042's spec ## Touch.
```

```
To widen scope: amend .work/tickets/T-0042/spec.md ## Touch (and plan.md), then
ask the user to type
```

```
`/crew:approve T-0042`. (scope.mode is block)
```

At the End of a Turn: The Completion Audit

The guard only sees the editing tools. A `sed -i`, a shell redirect or a formatter can still change files. So when the session stops, the completion audit compares the whole tree with the commit the ticket started from. It covers committed, staged, unstaged and untracked changes, and both ends of a rename. If any changed path is outside Touch, the audit blocks the stop and lists the paths in six lines or fewer. It never blocks the continuation it caused. `/crew:done` runs the same check:

```
python3 <crew>/hooks/scripts/completion_audit.py --check --ticket T-0042
```

The audit sees what git sees. Gitignored files, including everything under `.crew/`, and `.work/` are outside it.

Amending Scope

1. Edit `spec.md ## Touch` to add the path. Update `plan.md` if a step now changes it.
2. Run `crew_ticket.py validate --ticket <id>`.
3. Type `/crew:approve <id> yourself`.

Between steps 1 and 3 the approval is stale. The guard refuses edits outside the ticket directory until you approve again.

After Two Review Rounds: A Successor Plan

If a ticket uses both review rounds without an accepted receipt, it moves to `NEEDS_REPLAN`. Write a different plan and approve it. `approve reports review may continue`, and the ledger gives the new plan two fresh rounds. Approving the same plan again is refused and exits with status 3.

Modes

Set `scope.mode` in `.crew/config.json`:

Value	What happens
off	Default. Neither hook does anything.
report	Everything is allowed. Would-be refusals go to <code>.crew/guard.log</code> and appear as a system message.
block	Refusals block.
auto	report for the first ten tickets approved in the repository, then block.

If the config file does not parse, or `scope.mode` has an unknown value, crew treats it as `block` and says so.

Which Ticket Is Active

`crew_ticket.py activate --ticket <id>` sets the active ticket for the current worktree. `deactivate` clears it, and `active` prints it. Without a setting, crew uses the open ticket in `.work/INDEX.md` if `.work/tickets/<id>/` exists. With no active ticket, the edit guard only protects approval and ledger state, and the completion audit does nothing.